

Topics in Object-Oriented Methodology

Takuya Katayama
JAIST

1

Fundamental Concepts in Object-Oriented Methodology

4

Hi !

- Myself
 - Prof. at Japan Advanced Institute of Science and Technology
 - Working on formal and mathematical aspect of software engineering, especially
 - Object-Oriented Methodology
 - Software evolution
 - Fault-tolerant software
 - My family
 - One dog, One wife and two daughters

2

What is Object-Oriented Methodology ?

- Software developing methodology based on Object-Oriented Paradigm
 - Our world is a collection of collaborating agents/objects
 - Software has to be organized according to the structure of our world
 - It increases understandability and evolvability of the software

5

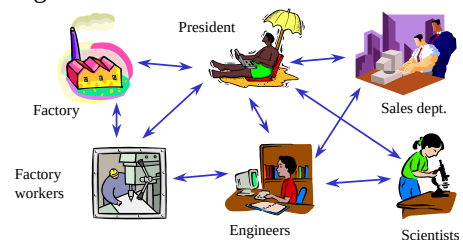
Agenda

- Fundamental Concepts in Object-Oriented Methodologies
 - UML
- Topics in OO Methodologies
 - Constraints and OCL
 - Proving Constraints

3

Object-Oriented Paradigm(1)

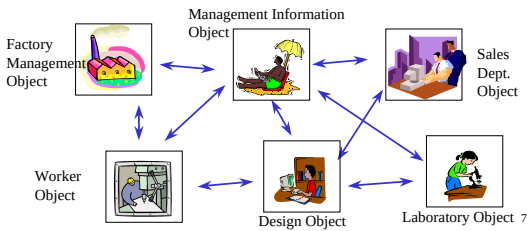
- Our World is a collection of collaborating agents



6

Object-Oriented Paradigm(2)

- Organize software according to the structure of the world



Description of the World

- How do we describe the world?
 - Class concept + Relations among classes
- Class as a set of “similar objects” in the world
 - Abstraction : {professor Shinoda, professor Tan, ...} → class “professor”
 - Instantiation : “professor” → professor Shinoda
 - Class concepts provides economy and reuse of thought and description.

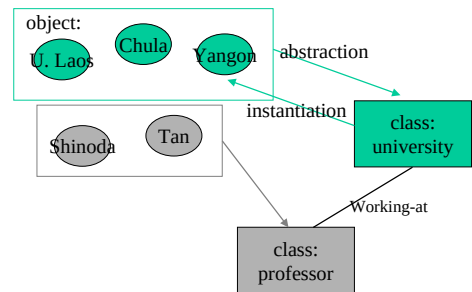
10

Computationally, Objects are...

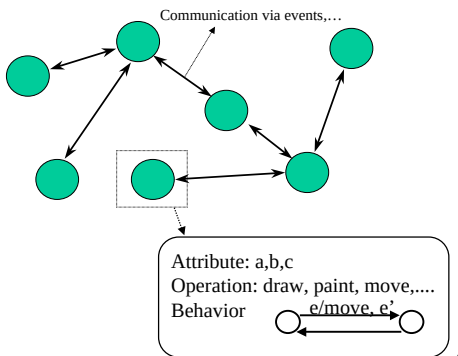
- Elements of the world to be described
 - Autonomous behavioral elements
- Object = Attributes + Operations + Behavior
 - Attributes: Local data of object
 - Operations: For changing/referencing the attributes
 - Behavior: Upon receiving stimulus(event), perform operations, changes its states and sends out events
 - Usually, modeled as statecharts or event-driven program
- Object is encapsulated.

8

Objects and Classes



11



9

Relationships Among Classes

- Class Hierarchy, Inheritance, “is a”
 - Generalization/Specialization
 - Mammal < Monkey < Human
- Composition, Aggregation, “has a”
 - Automobile = Body + Wheels + Steering + Engine
- Association, a general relation between classes
 - Employ— (works at) —Company
- Dependency, Realization,...

12

Too Major Issues in Object-Oriented Methodology

- Object-Oriented Analysis/Design
 - BOOCH, OMT, UML, Catalysis methods
 - Constraints, Formal Approach, Analysis Patterns, Unified Process, ...
- Object-Oriented Programming
 - OO languages :Smalltalk, C++, Java
 - Design Patterns, Frameworks , Class Libraries
 - ...

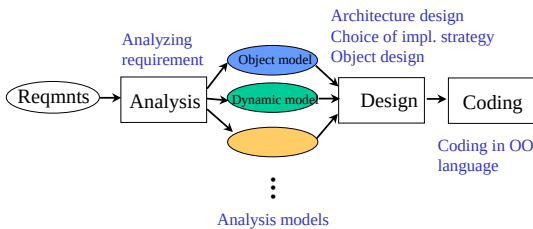
13

UML

- Notations to describe every aspects of software development
 - From business systems to industrial embedded systems
 - Becoming a standard notation
- Unified notations in Booch method, OMT method(Rumbaugh), OOSE method (Jacobson)
- Becoming a standard language or culture of software engineer

16

Typical Object-Oriented Development Process



14

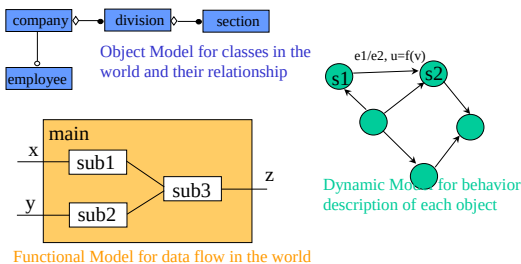
UML- Eight Diagrams and One Language

- Use Case Diagram
- Class Diagram/Object Diagram
- State Diagram
- Sequence Diagram
- Collaboration Diagram
- Activity Diagram
- Component Diagram
- Deployment Diagram
- OCL(Object Constraint Language)

17

Multiple Analysis Models

- Description of a single model from multiple views which are almost independent



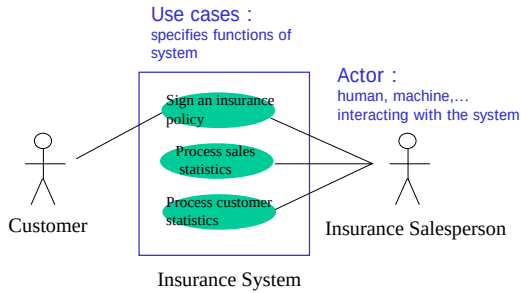
15

Use Case Diagram

- Use Cases describe a rough sketch of functions or “usage” of the system looked from the actors outside the system.
- System functions are described by a set of use cases.

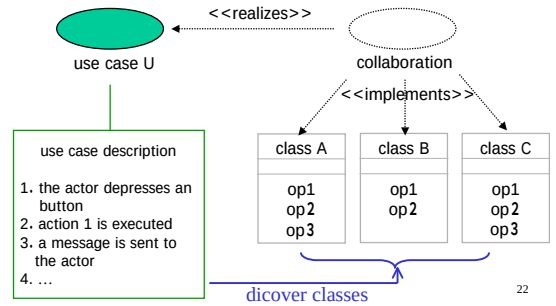
18

Use Case Diagram



19

In OO development, use cases are used to find objects. Functions represented by the use cases are realized as collaborations among the objects.



22

Description of Use Case

- By sentences or activity diagrams
 - Purpose of use case
 - How the use case is initiated
 - Message flow between actors and use case
 - Alternative flow
 - conditional/exceptional flow
 - do not go into too much details
 - Utilizing of other use cases : use case call
 - How the use case is terminated

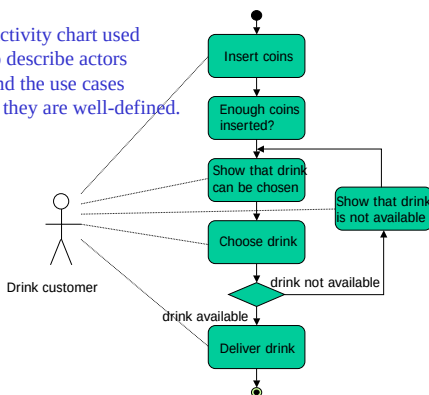
20

Class Diagrams

- Classes in the system + Relations
- Relations among classes
 - Inheritance
 - Aggregation
 - Association
 - Dependency
 - Realization

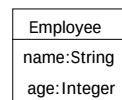
23

Activity chart used to describe actors and the use cases if they are well-defined.

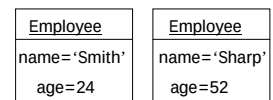


21

Class and Object



Class Employee

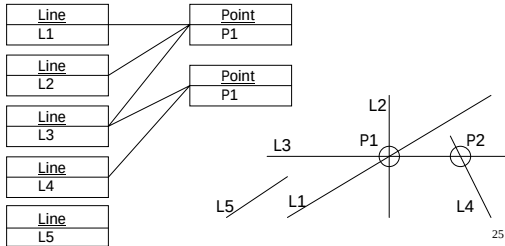


Objects instantiated from the class Employee

24

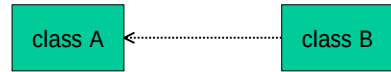
Association and Multiplicity

Line	2..*	0..*	Point
name	Intersect		name



25

Dependency



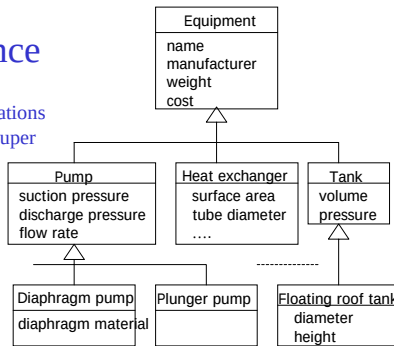
- Utilize class A in class B
 - B has A as a parameter
 - B accesses data in A
 - B uses operations A
 - Change of A affects B

28

Inheritance

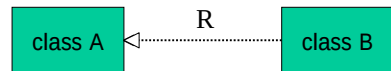
Attributes and operations are inherited from super classes

Diaphragm pump
name=P101
manufacture=Simplex
...
suct pressure=1.1 atm
...
diaph material=Teflon



26

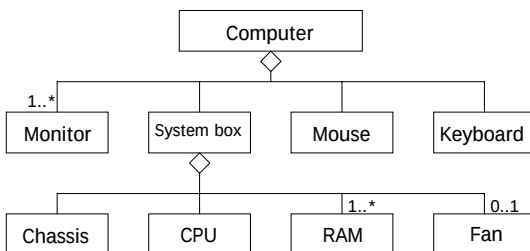
Realization



- Class B is a realization of A
 - B is more concrete
 - A is more abstract
 - R: mapping $B \rightarrow A$

29

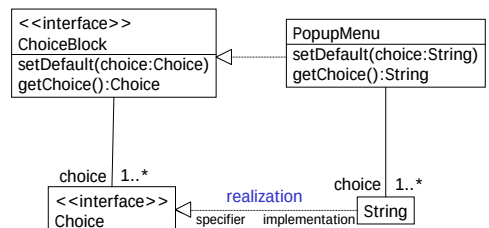
Aggregation



27

Interface and Realization

- Interface is a special class with abstract operation. It is used with classes which realize/implement them.



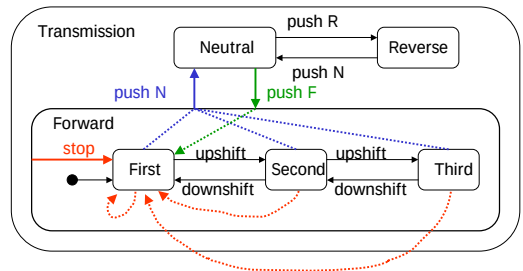
30

Dynamical Views

- State Diagram/Statechart
 - Specify dynamics of individual object
- Sequence Diagram
 - Event sequence observed in collaborating objects
- Collaboration Diagram
 - Different notation of sequence diagram
- Activity Diagram
 - Action + Control structure
 - Flowchart

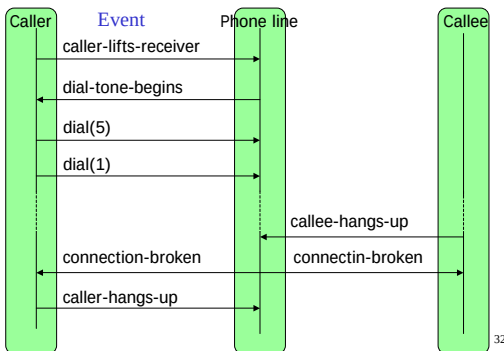
31

Generalized State



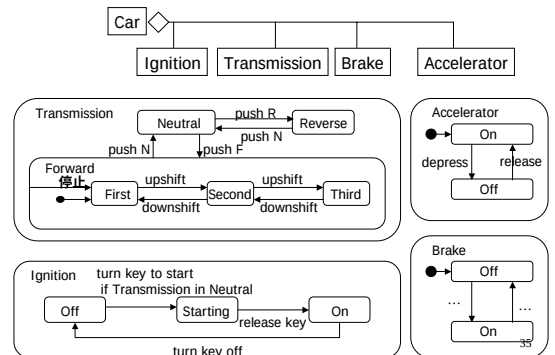
34

Telephone System



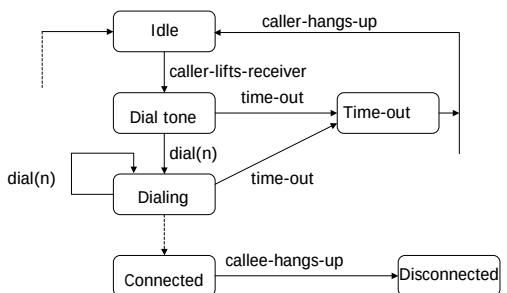
32

Aggregation and Concurrent State Diagram



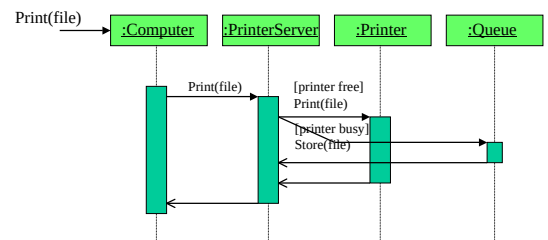
35

State Diagram for "Phone line"



33

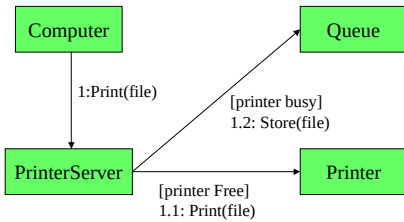
Sequence Diagram



- Shows possible sequence(s) of messages among objects
- Inter-object behavior

36

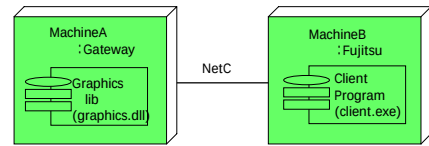
Collaboration Diagram



37

Deployment Diagram

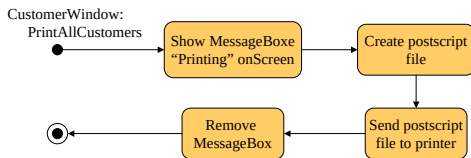
- Machines, programs and connections



40

Activity Diagram

- Shows relationship among actions
 - Intra-object actions
 - Inter-object actions



38

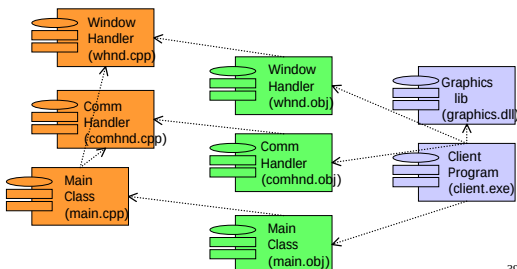
Topics in OO Methodologies

- (1) Object Constraint Language OCL
- (2) Formal Approach to OO Analysis

41

Component Diagram

- Software components(source, binary and executable) and their relationship



39

Constraints and OCL

- Constraints
 - Description of systems and services using their properties
 - Usually, constraints are described by the results of the services rather than the procedure to realize them.

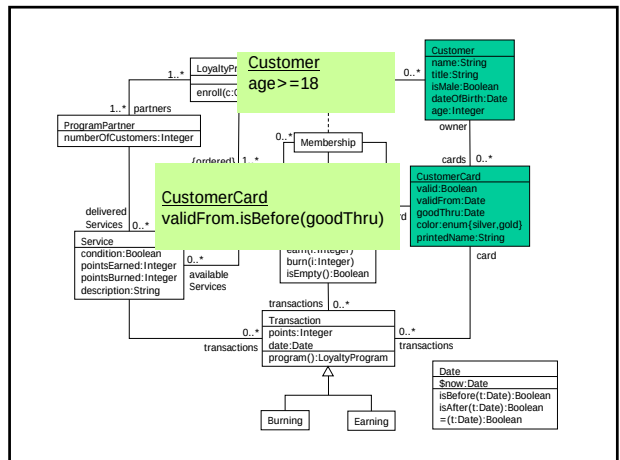
What rather than How.

“Any letter posted until 6:00 pm has to be delivered in the next working day”

42

- Constraints in UML
 - Constraints on attribute values in class diagram
 - Definition of operations in classes by pre/post-conditions
- Benefit of constraints
 - Provide means to attach semantic information to UML class diagrams
 - Allow declarative definition of behavior
- OCL : standard constraint language in UML
 - Based on first order predicate logic

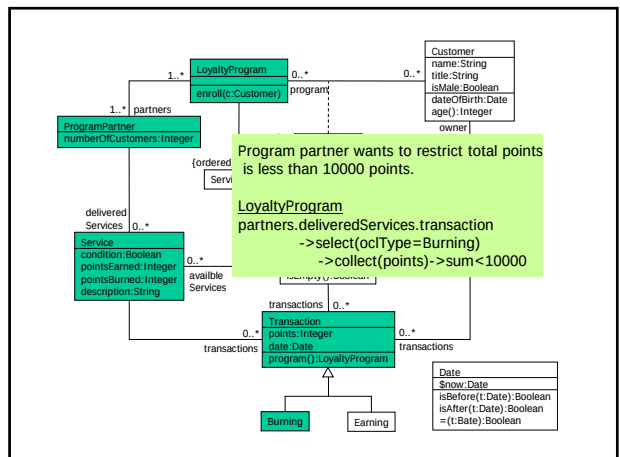
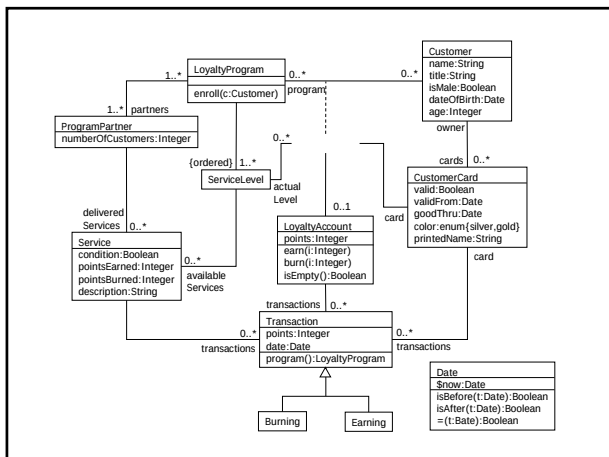
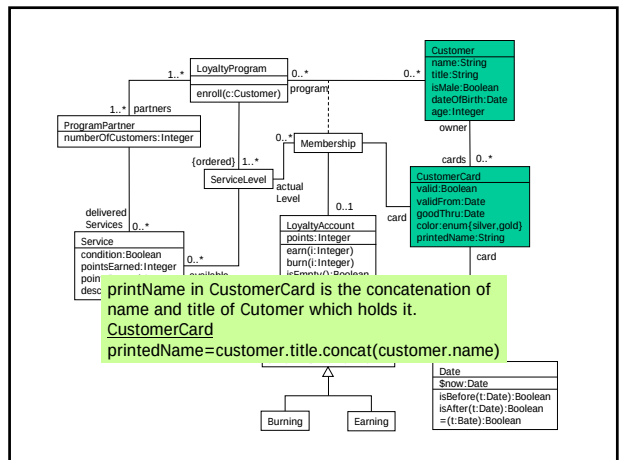
43



“Royal Loyal” Company

- Mileage Handling company
- LoyaltyProgram
 - Bonus point
 - Air flight mileage
 - Deduction
- ProgramPartner: a company offering its customer a membership in a loyalty program
- Customer owns CustomerCard

44



- Defining operations by Pre-, Postcondition
 - precondition: specifies condition when the operation could be triggered.
 - postcondition: specifies condition which holds after the operation.

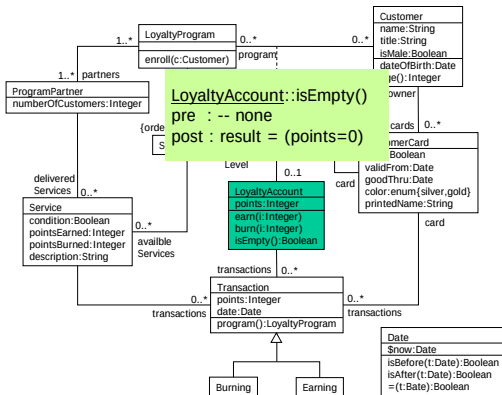
precondition P postcondition

49

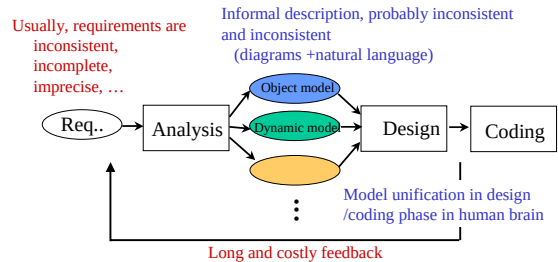
Formal OO Methodology

- Problems of Current OO Methodologies
 - Formality is very low and effective computer support is difficult, especially in the analysis phase.
 - Quality of the analysis models is not good, which determines the overall quality of final products.
- So-called formal methods do not consider practices of OO methodologies seriously.
 - Object-orientation of formal methods
- Formalization of practical OO methodologies is needed.

52



Current OO Methodology



53

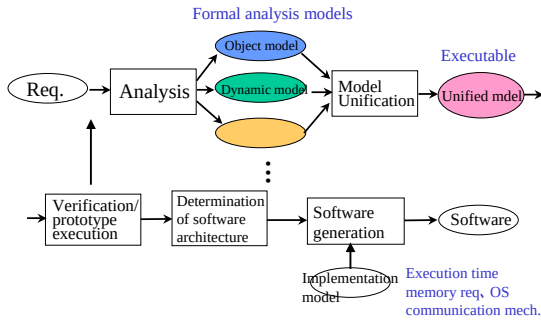
A Formal Approach to Object-Oriented Analysis

- Aiming at Formalized OO Methodology
 - Formal analysis models
 - From multiple view points
 - Consistency among analysis models, construction of a unified model
 - Machine assistance in Validation and Verification of analysis models
 - Prototype execution and verification by theorem proving techniques
 - Transformation from the unified models to software architecture
 - Software architecture + implementation model => source codes

51

54

Formal OO Methodology

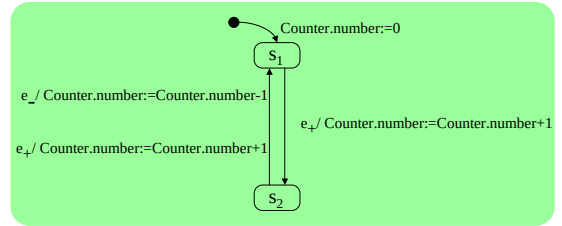


55

Counter
number

Class Counter

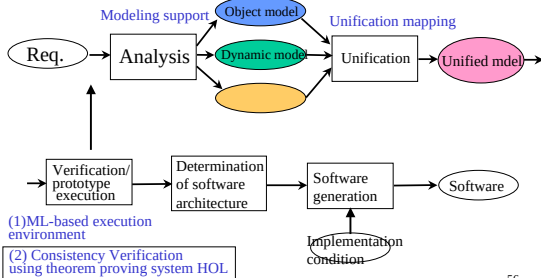
Dynamic model for Counter (statechart after unification)



Formal OO Methodology

(what we have done)

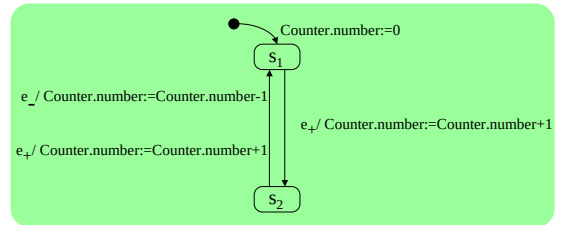
Formal analysis models in ML



56

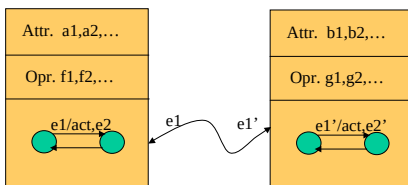
Counter
number

constraint: "Counter.number ≥ 0"



Consistency between Object Model and Dynamic Model

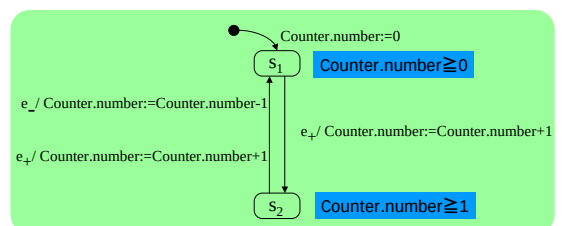
- Verify that constraints among attributes in object models are maintained for any behavior of objects.



57

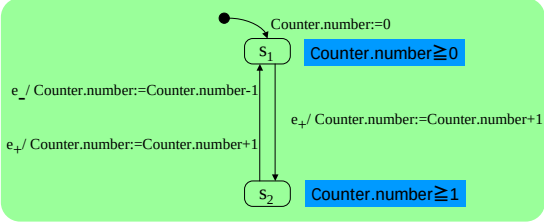
Counter
number

Prove "Counter.number ≥ 0"



Counter
number

Prove "Counter.number ≥ 0"



- (1) Counter.number ≥ 0 at S1 ⇒ Counter.number ≥ 1 at S2
i.e. Counter.number ≥ 0 ⇒ (Counter.number + 1) ≥ 1
- (2) Counter.number ≥ 1 at S2 ⇒ Counter.number ≥ 0 at S1
i.e. Counter.number ≥ 1 ⇒ (Counter.number ± 1 ≥ 0)

Local Invariant Axiom

$$\forall o \in I(c). \forall LAS_1 \dots LAS_n.$$

$$\text{valid}[o](LAS_1, \dots, LAS_n) \Rightarrow \bigwedge_{1 \leq i \leq n} LAS_i(\sigma[o][s_i])$$

where

$$\text{valid}[o](LAS_1, \dots, LAS_n) \Leftrightarrow$$

$$LAS_1(v) \wedge \bigwedge_{t \in T} LAS_{s(t)}(\sigma[o][s(t)] \wedge \text{cond}_t(\sigma[o][s(t)]) \Rightarrow LAS_{d(t)}(\sigma'_t[o][d(t)])$$

v : list of initial values of attributes

cond_t : transition condition of a transition $t \in T$

T : the set of transitions of the object o

$\sigma[o][s]$: list of attribute values of o at the state s

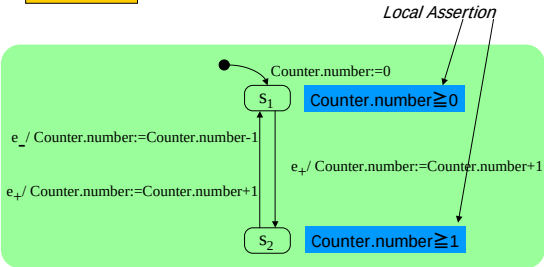
$s(t)$, $d(t)$: source and destination states of the transition t

$\sigma'_t[o][s]$: list of attribute values changed by actions of t

64

Counter
number

Counter.number ≥ 0 : Global Assertion



- (3) Counter.number ≥ 0 ∧ Counter.number ⇒ Counter.number ≥ 0

Global Invariant Axiom

$$\forall o \in I(c). \forall GA.$$

$$\bigwedge_{1 \leq i \leq n} GA(\sigma[o][s_i]) \Rightarrow (\forall s. GA(\sigma[o][s]))$$

65

Axioms

- Six Axioms:
 - Local invariant axiom
 - Global invariant axiom
 - Event output axiom
 - Event communication axiom
 - Inheritance axiom
 - Aggregation axiom

63

Event Output Axiom

$$\forall o \in I(c). o?e@s \wedge \text{cond}_t(\sigma[o][s]) \Rightarrow \bigwedge_{1 \leq k \leq l} o!e_k@s'$$

s, s' : source and destination states of a transition t

e : input event of the transition t

$e_1, \dots, e_k$: output event of t

$o?e@s$: expresses that event e could be received at the state s of object o

$o!e@s$: expresses that an event e could be sent at the state s of object o

66

Event Communication Axiom

$(o, o') \in \text{Link}$, that is, if there is a link between objects o and o' ,

$$\forall P. o!e@s \wedge P(\tau[o][s](a_1), \dots, \tau[o][s](a_n)) \Rightarrow o'?e@s' \wedge P(\tau[o'][s'](a_1), \dots, \tau[o'][s'](a_n))$$

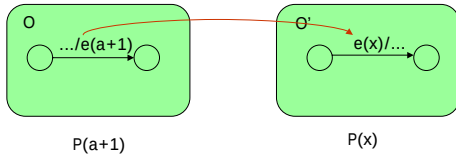
P : property about event attributes $a_1, \dots, a_n$ event e
 $\tau[o][s](a_k)$: values of $a_1, \dots, a_n$ at the state s

67

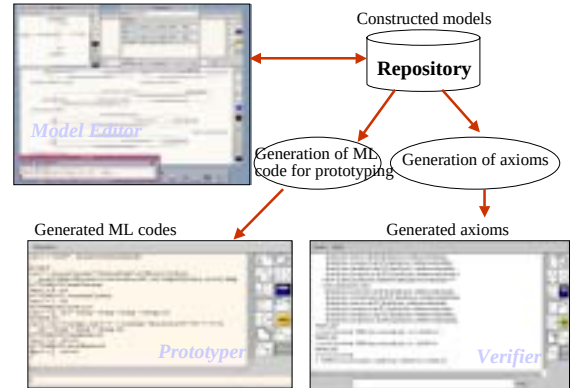
F-Developer Environment

- *F-Developer*
 - Model Editor for constructing analysis models graphically
 - F-Verifier: for verifying constructed models using HOL
 - HOL: Higher order predicate logic prover developed at Cambridge Univ.
 - Proof-checker rather than automatic prover
 - F-Prototyper: for prototype execution of the models

70



68

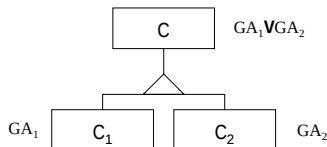


Inheritance Axiom

Let a class c be a super class of $c_1, \dots, c_n$, and o and o_i be instantiated from c and c_i .

$$\forall GA_1 \dots GA_n$$

$$\bigwedge_{1 \leq i \leq n} \forall s_i. GA_i(\sigma[o_i][s_i]) \Rightarrow \bigvee_{1 \leq i \leq n} \forall s. GA_i(\sigma[o][s])$$



69

Generation of Axioms

- Axioms are introduced into HOL as its theory modules, which are type/term constants, axioms or definitions.

Type constants:
 Counter 0
 EventID 0
 StateID 1
 ObjectID 1
 Term constants:
 s1 (Prefix) :Counter StateID
 s2 (Prefix) :Counter StateID
 Counter_number (Prefix):
 Counter ObjectID -> Counter StateID -> num
 inc(Prefix) :num -> num
 Counter_LA_s1 (Prefix) :num -> bool
 Counter_LA_s2 (Prefix) :num -> bool
 Counter_GA (Prefix) :num -> bool
 valid_Counter (Prefix):
 e_plus (Prefix) :EventID
 e_minus (Prefix) :EventID
 link (Prefix): 'a ObjectID # 'b ObjectID -> bool

send (Prefix):
 'a ObjectID # EventID # 'a StateID -> bool
 receive (Prefix):
 'a ObjectID # EventID # 'a StateID -> bool
 obj (Prefix): Counter ObjectID
 ...
 Axioms:
 AX-L1_Counter- lo LA_s1 LA_s2.
 valid_Counter (o,LA_s1,LA_s2) ==>
 LA_s1(Counter_number o s1) & LA_s2 (Counter_number o s2)
 AX-G1_Counter- ...
 ...
 Definitions:
 Counter_inc_DEF- In.inc n = n+1
 Counter_LA_s1_DEF- lx. Counter_LA_s1 x = 0<=x
 Counter_LA_s2_DEF- lx. Counter_LA_s2 x = 1<=x
 Counter_GA_DEF- lx. Counter_GA x = 0<=x
 ...

Proof in HOL

```
- ADD_MONO_LESS_EQ;
val it = |> lm n p. m + n <= m + p = n <= p : thm
- ADD_0;
val it = |> lm. m + 0 = m : thm
- SPEC (~> Counter_number obj s1 ~>)
  (SPEC (~> 0 ~>) ADD_MONO_LESS_EQ);
val it = |> 1 + 0 <= 1 + Counter_number obj s1 =
  0 <= Counter_number obj s1 : thm
- val lemma1 = snd (EQ_IMP_RULE it);
val lemma1 =
|> 0 <= Counter_number obj s1 ==>
  1 + 0 <= 1 + Counter_number obj s1 : thm
- SPEC (~> 1 ~>) ADD_0;
val it = |> 1 + 0 = 1 : thm
- val lemma2 = REWRITE_RULE [it] lemma1;
val lemma2 = |> 0 <= Counter_number obj s1 ==>
  1 <= 1 + Counter_number obj s1
```

```
- val r1 =
  SYM (SPEC (~> Counter_number obj s1 ~>)
    Counter_LA_s1_DEF);
val r1 =
|> 0 <= Counter_number obj s1 =
  Counter_LA_s1 (Counter_number obj s1) : thm
- val r2 =
  SYM (SPEC (~> 1 + Counter_number obj s1 ~>)
    Counter_LA_s2_DEF);
val r2 =
|> 1 <= 1 + Counter_number obj s1 =
  Counter_LA_s2 (1 + Counter_number obj s1) : thm
- val step1 = REWRITE_RULE [r1, r2] lemma2;
val step1 =
|> Counter_LA_s1 (Counter_number obj s1) ==>
  Counter_LA_s2 (1 + Counter_number obj s1) :
  thm
```

73

Lessons Learned

- Proof requires long steps, as we need to prove
 - Facts about primitive data such as integer, list, ...
 - Also, inference rules are primitive.
- To make it acceptable to “software engineer”, we need
 - Abstract domain libraries
 - Theorems about ‘Banking System’, ‘Hotel’, ...
 - Reuse of proof steps
 - Domain specific proof tactics
 - Proof tactics for ‘Banking System’, ...

76

Lessons Learned

- Found that formal verification is useful in developing correct analysis models
 - Even for the small example we tried, errors were found by the proving assertions.
 - In the analysis phase, errors could come in from incomplete and inconsistent requirement.
 - HOL style interactive proof making process could be considered as a process of gradually understanding / constructing analysis models.

74

Lessons Learned

- Higher Order Property
 - If you can quantify predicates in your expression, your logic is higher order logic.
$$\forall p \forall x [P(x) \Rightarrow x=0]$$
- Higher order logic is hard to handle in automatic proving, but it is very useful in describing analysis model.
 - It could be used to express general knowledge of the domain.
 - Reuse/economy of description

75